Name – Mohit Gabra
Roll no 2401027
Class: TE
Batch T1-T12

Experiment No – 01

LO2 – use git effectively for version control source code management and colabrative team development

Theory:

Git commands are instructions used to interact with the Git version control system. These commands allow developers to create repositories, track changes, save versions of code, manage branches, and control project history.

The commonly used Git commands can be divided into different operations:

1. Repository Management Commands

`git init`

- Creates a new Git repository in the current folder.
- Initializes Git tracking for a project.

`git --version`

- Displays the installed Git version.

---

2. Configuration Commands

`git config`

- Used to set and view Git user information.

Examples:

```
git config user.name "Name"
git config user.email "email"
```

It helps Git identify the author of commits.

Name – Mohit Gabra
Roll no 2401027
Class: TE
Batch T1-T12

3. File Tracking Commands

`git status`

- Shows the current state of files.
- Displays modified, untracked, and staged files.

`git add`

- Adds files to the staging area before committing.

Example:

```
git add .
```

`git rm`

- Removes a file from the working directory and Git tracking.

Example:

```
git rm style.css
```

---

4. Commit and Version Commands

`git commit`

- Creates a permanent snapshot of staged changes.

Example:

```
git commit -m "Added homepage"
```

`git log`

- Displays the history of commits.

Example:

```
git log --oneline
```

`git show`

- Shows detailed information about a specific commit, including changes made.

Name – Mohit Gabra
Roll no 2401027
Class: TE
Batch T1-T12

---

5. Branch Management Commands

`git branch`

- Creates and displays branches.

Example:

```
git branch feature-page
```

`git switch`

- Changes from one branch to another.

Example:

```
git switch feature-page
```

`git merge`

- Combines changes from one branch into another.

---

6. Change Management Commands

`git diff`

- Displays differences between current files and previous versions.

`git restore`

- Restores files to their previous state or removes unwanted changes.

`git stash`

- Temporarily saves unfinished changes without committing them.

---

7. Repository Information Commands

`git ls-files`

Name – Mohit Gabra
Roll no 2401027
Class: TE
Batch T1-T12

- Displays files currently tracked by Git.

`git branch -a`

- Shows all available branches.

1. Check Git version

```
L15@LAB05-008 MINGW64 ~/Myproject (main)
$ git --version
git version 2.55.0.windows.2
```

2. Check Git configuration

```
L15@LAB05-008 MINGW64 ~/Myproject (main)
$ git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=openssl
http.sslcainfo=C:/Program Files/Git/mingw64/etc/ssl/certs/ca-bundle.crt
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=master
user.name=Your Name
user.email=your.email@example.com
core.repositoryformatversion=0
core.filemode=false
core.bare=false
core.logallrefupdates=true
core.symlinks=false
core.ignorecase=true
remote.origin.url=https://github.com/Mohit/Myproject.git
remote.origin.fetch=+refs/heads/*:refs/remotes/origin/*
```

3. Set your name

```
L15@LAB05-008 MINGW64 ~/Myproject (main)
$ git config user.name "Mohit"
```

4. Set your email

Name – Mohit Gabra
Roll no 2401027
Class: TE
Batch T1-T12

```
L15@LAB05-008 MINGW64 ~/Myproject (main)
$ git config user.email "mohit@gmail.com"
```

5. Check current folder

```
L15@LAB05-008 MINGW64 ~/Myproject (main)
$ pwd
/c/Users/L15/Myproject
```

6. List files

```
L15@LAB05-008 MINGW64 ~/Myproject (main)
$ ls
README.md
```

7. Create a file

```
L15@LAB05-008 MINGW64 ~/Myproject (main)
$ touch index.html
```

8. Check repository status

```
L15@LAB05-008 MINGW64 ~/Myproject (main)
$ git status
On branch main
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        index.html

nothing added to commit but untracked files present (use "git add" to track)
```

9. Add a file to staging

```
L15@LAB05-008 MINGW64 ~/Myproject (main)
$ git add index.html
```

10. Add all files

```
L15@LAB05-008 MINGW64 ~/Myproject (main)
$ git add .
```

11. Create your first commit

Name – Mohit Gabra
Roll no 2401027
Class: TE
Batch T1-T12

```
$ git commit -m "first commit"
[main 3351c27] first commit
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 index.html
```

12. View commit history

```
$ git log
commit 3351c273c90fe441d325a22ff57f51b88a8e03f1 (HEAD -> main)
Author: Mohit <mohit@gmail.com>
Date:   Tue Jul 14 11:20:24 2026 +0530

    first commit

commit 80f8fb964ef7a021d4d95839bf993cdc30094764 (login-page)
Author: Your Name <your.email@example.com>
Date:   Tue Jul 14 10:17:59 2026 +0530

    First commit
```

13. View short commit history

```
L15@LAB05-008 MINGW64 ~/Myproject (main)
$ git log --oneline
3351c27 (HEAD -> main) first commit
80f8fb9 (login-page) First commit
```

14. See changed files

```
L15@LAB05-008 MINGW64 ~/Myproject (main)
$ git status
On branch main
nothing to commit, working tree clean
```

15. Create a branch

```
L15@LAB05-008 MINGW64 ~/Myproject (main)
$ git branch test-branch
```

16. See all branches

```
L15@LAB05-008 MINGW64 ~/Myproject (main)
$ git branch
  login-page
* main
  test-branch
```

Name – Mohit Gabra
Roll no 2401027
Class: TE
Batch T1-T12

17. Switch to a branch

```
L15@LAB05-008 MINGW64 ~/Myproject (main)
$ git switch test-branch
Switched to branch 'test-branch'
```

18. Create and switch to a new branch

```
L15@LAB05-008 MINGW64 ~/Myproject (test-branch)
$ git switch -c feature-page
Switched to a new branch 'feature-page'
```

19. Create another file

```
L15@LAB05-008 MINGW64 ~/Myproject (feature-page)
$ touch style.css
```

20. Check differences

```
L15@LAB05-008 MINGW64 ~/Myproject (feature-page)
$ git diff
```

21. Remove a file from Git tracking

```
L15@LAB05-008 MINGW64 ~/Myproject (feature-page)
$ git rm style.css
rm 'style.css'
```

22. Undo staged changes

```
L15@LAB05-008 MINGW64 ~/Myproject (feature-page)
$ git restore --staged index.html
```

23. Temporarily save changes

```
L15@LAB05-008 MINGW64 ~/Myproject (feature-page)
$ git stash
Saved working directory and index state WIP on feature-page: 4c9176d Added CSS file
```

Name – Mohit Gabra
Roll no 2401027
Class: TE
Batch T1-T12

24.View all Git branches

```
L15@LAB05-008 MINGW64 ~/Myproject (feature-page)
$ git branch -a
* feature-page
  login-page
  main
  test-branch
```

25. View Git information

```
L15@LAB05-008 MINGW64 ~/Myproject (feature-page)
$ git show
commit 4c9176da52342904b85d8f9806dc1405f08b6b5a (HEAD -> feature-page)
Author: Mohit <mohit@gmail.com>
Date:   Tue Jul 14 11:29:49 2026 +0530

    Added CSS file

diff --git a/style.css b/style.css
new file mode 100644
index 0000000..e69de29
```

Name – Mohit Gabra
Roll no 2401027
Class: TE
Batch T1-T12

Conclusion

The Git commands practiced help in managing and controlling software project versions. Commands like git init, git add, and git commit create and save project versions, while commands like git branch, git switch, and git merge help manage different development paths. Commands such as git status, git log, and git show allow developers to monitor and understand project history.